Reviewer Pack — Minimal Rank of Quadratic Forms and Resolution Proof Width

Entry 1ecbca5d4eed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 14:33:32 UTC

Minimal Rank of Quadratic Forms and Resolution Proof Width

Entry ID: `1ecbca5d4eed` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 14:33:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> The minimal rank of the quadratic form associated with a Tseitin formula φ_G is linearly correlated with its resolution proof width w(φ_G), such that min_rank(Q(φ_G)) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> Quadratic forms provide a way to encode boolean functions, and their ranks can reflect the complexity of the underlying function. Resolution proof complexity measures the difficulty of solving a problem using the resolution algorithm. This conjecture suggests that these two measures are correlated, potentially revealing new insights into the structure of boolean functions and their computational complexity.

Taxonomy category: `AlgebraicGeometryQuadraticForms` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8e298c3ffa85f4f7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n from 1 to 40, the correlation coefficient between min_rank(Q(φ_G)) and w(φ_G) exceeds 0.9 across at least 30 independent seeds and their mean is within ±2 of a linear fit with a slope of 1, or is falsified if any seed does not meet these criteria.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate non-pivot elements
        pivot = matrix[i][i]
        for j in range(i, n):
            matrix[i][j] /= pivot
        for k in range(n):
            if k != i:
                factor = matrix[k][i]
                for j in range(i, n):
                    matrix[k][j] -= factor * matrix[i][j]

    # Back-substitution to find the rank
    rank = 0
    for row in matrix:
        if any(row[i] != 0 for i in range(n)):
            rank += 1
    return rank

def quadratic_form(literals, clauses):
    n = len(literals)
    Q = [[0] * n for _ in range(n)]

    for clause in clauses:
        literals_in_clause = [l for l in clause if l != 0]
        if not literals_in_clause:
```

```
            continue
        i = int(abs(literals_in_clause[0]) - 1) % n
        j = int(abs(literals_in_clause[1]) - 1) % n
        Q[i][j] += 1
        Q[j][i] += 1

    return gaussian_elimination(Q)

def tseitin_formula(n):
    literals = [f'x{i+1}' for i in range(n)]
    clauses = []

    # Clause: x1 ∨ ¬x2 ∨ ... ∨ ¬xn
    clause = [-i for i in range(1, n+1)]
    clauses.append(clause)

    # Clauses: xi ∨ ¬xi -> ¬xi ∨ ¬xi (tautology)
    for i in range(n):
        clause = [i+1, -(i+1), -(i+1)]
        clauses.append(clause)

    return literals, clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in range(5, 41):
        literals, clauses = tseitin_formula(n)
        min_rank = quadratic_form(literals, clauses)
        resolution_width = len(clauses)  # Simplified for demonstration

        results.append({
            "n": n,
            "min_rank": min_rank,
            "resolution_width": resolution_width
        })

    if not results:
        return {
            "metric_name": "min_rank",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    min_ranks = [r["min_rank"] for r in results]
    widths = [r["resolution_width"] for r in results]

    correlation_coefficient = sum((min_ranks[i] - mean_min_ranks) * (widths[i] - mean_widths) for

    mean_min_ranks = sum(min_ranks) / len(min_ranks)
    mean_widths = sum(widths) / len(widths)
```

```
    if correlation_coefficient < 0.9:
        return {
            "metric_name": "min_rank",
            "metric_value": correlation_coefficient,
            "instances_tested": len(results),
            "n_max": max(r["n"] for r in results),
            "conjecture_holds": False,
            "counterexample": f"Correlation coefficient {correlation_coefficient} < 0.9"
        }

    return {
        "metric_name": "min_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(r["n"] for r in results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {seed} {trial_result}")

        if not "metric_value" in trial_result or trial_result["conjecture_holds"] is False:
            break

    results = [run_trial(seed) for seed in seeds]

    supported_count = sum(1 for r in results if r["conjecture_holds"])
    support_fraction = supported_count / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.9\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b1e0bd3.py", line 132, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b1e0bd3.py", line 103, in run_trial
```

```
    correlation_coefficient = sum((min_ranks[i] - mean_min_ranks) * (widths[i] -
mean_widths) for i in range(len(results))) / math.sqrt(sum((min_ranks[i] - mean_min_ranks)**2 for i in
mean_widths)**2 for i in range(len(results))))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0b1e0bd3.py", line 103, in <genexpr>
    correlation_coefficient = sum((min_ranks[i] - mean_min_ranks) * (widths[i] -
mean_widths) for i in range(len(results))) / math.sqrt(sum((min_ranks[i] - mean_min_ranks)**2 for i in
mean_widths)**2 for i in range(len(results))))
                                            ^^^^^^^^^^^^^^
NameError: cannot access free variable 'mean_min_ranks' where it is not associated with a value in encl
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Review and debug the test code to ensure it can run successfully and produce the required data.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19256
2	propose	ollama_remote	glm4:latest	0	0	18156
3	preregistration	ollama_remote	glm4:latest	0	0	9293
4	novelty	ollama_remote	glm4:latest	0	0	8404
5	novelty	ollama_remote	glm4:latest	0	0	8568
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16665
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20703
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11962
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15875
10	judge	ollama_remote	glm4:latest	0	0	11754

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 140637 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1ecbca5d4eed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/1ecbca5d4eed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1ecbca5d4eed.tar.gz` (if generated)